

Bharosa: Trusted Search for Families Who Can't Afford Care

Track T1: Retrieval-Augmented Generation and trustworthy answers

Parth Agarwal(2410110234), Paridhi Kumar(2410110233),

Kushagra Bansal(2410110184) & Ishanvi Singh(2410110150)

[Github Repo](#)

Abstract. When someone in a family falls sick, three questions arrive together: can this medicine cost less, is there a government scheme that can help, and can I trust this message asking me to pay a fee? Bharosa ("trust") is a search system for these three questions with one rule: *retrieval first, generation last, and never guess*. A polite crawler collects official `.gov.in` pages, which are split into zones and indexed with an inverted, positional and parametric index written from scratch. Queries are ranked with Inc.Itc cosine, heap top-K and a net score that adds source quality $g(d)$. An LLM may write an answer only from the retrieved zones, every claim must cite a zone, a checker flags unsupported claims, and the system refuses when retrieval confidence is low. Medicine search uses no LLM: character n-gram matching finds the brand and a parametric filter keeps only rows with the same salt, strength and dosage form. On 15 verified queries over 235,155 records our method has the best precision at 5 for substitutable medicines (0.40, against 0.24 for Soundex and 0.11 for exact match), but it abstains on 8 of 15 queries. Scheme retrieval and RAG quality are not yet measured on judged data, and we say so plainly.

1. Problem and Track Relevance

1.1 The problem and the user

Our user is a family member, not a clinician. They type the way people text: `pantocid 40 sasta kya milega` ("is there something cheaper than Pantocid 40"), with spelling mistakes and Hindi words in Latin script. The information they need exists but is hard to reach. Medicine equivalence is buried in about a quarter of a million product records, where two similarly spelled brands may contain different salts. Scheme rules sit on long, formal government pages, and the user wants one part of the page, such as eligibility or the documents required. Forwarded messages such as *"pay Rs 500 to activate free health card"* target exactly these families.

A wrong answer here costs money or health, so a fluent but unsupported answer is worse than no answer. That is the design constraint for the whole system and the reason for its three modules.

Module	User types	System returns	LLM used?
Medicine finder	<code>pantocid 40 sasta kya milega</code>	Brands with the same salt, strength and dosage form, ranked by brand similarity, with MRP	No. Pure IR
Scheme finder	<code>health insurance for sanitation workers</code>	Ranked zones from official pages, then a short answer in which every claim cites a zone	Yes, over retrieved zones only
Claim checker	<code>pay Rs 500 to activate free health card</code>	Supported, contradicted or insufficient evidence, with the official line shown	Verdict is rule-based; LLM may only reword it

Table 1. The three modules of Bharosa.

1.2 Why Track T1

T1 asks for *"a RAG system where the retriever is a real IR component that you can inspect, and where every generated claim can be traced to a ranked source."* Each part of that sentence is a design decision in Bharosa:

- **Chunking is a "what is a document" decision.** We index page *zones* (eligibility, benefits, documents, how to apply), not whole pages, so a retrieved unit is small enough to cite.
- **The retriever is ours and inspectable.** The inverted index, Inc.Itc weighting, cosine and heap top-K are our own code, and `--verbose` prints postings, term weights and scores.
- **Every claim is traced and checked.** The LLM must return claims with zone identifiers, and a citation checker scores each claim against the zone it cites.
- **It refuses.** If the top net score is below a threshold, the LLM is never called.
- **Where generation is unsafe, it is absent.** Medicine answers come directly from ranked records.

The crawler (a T4 idea) and Hinglish handling (a T5 idea) are supporting parts. They build the corpus and clean the query that the trustworthy-answer pipeline depends on, but they are not our main claim.

1.3 Papers referred

The indexing and ranking core follows the vector space model [1] as presented in the course text [2]; BM25 [3] is our out-of-syllabus baseline. The answer layer follows retrieve-then-generate [4], and our citation requirement is motivated by work showing that LLM answers with citations are often only partly supported by the cited passages [5]. Near-duplicate detection uses shingling and resemblance [6]. The frontier design follows Mercator's separation of priority from per-host politeness [7], and the adaptive recrawl policy is motivated by work on page refresh policies [8].

2. How We Used IR

2.1 Pipeline

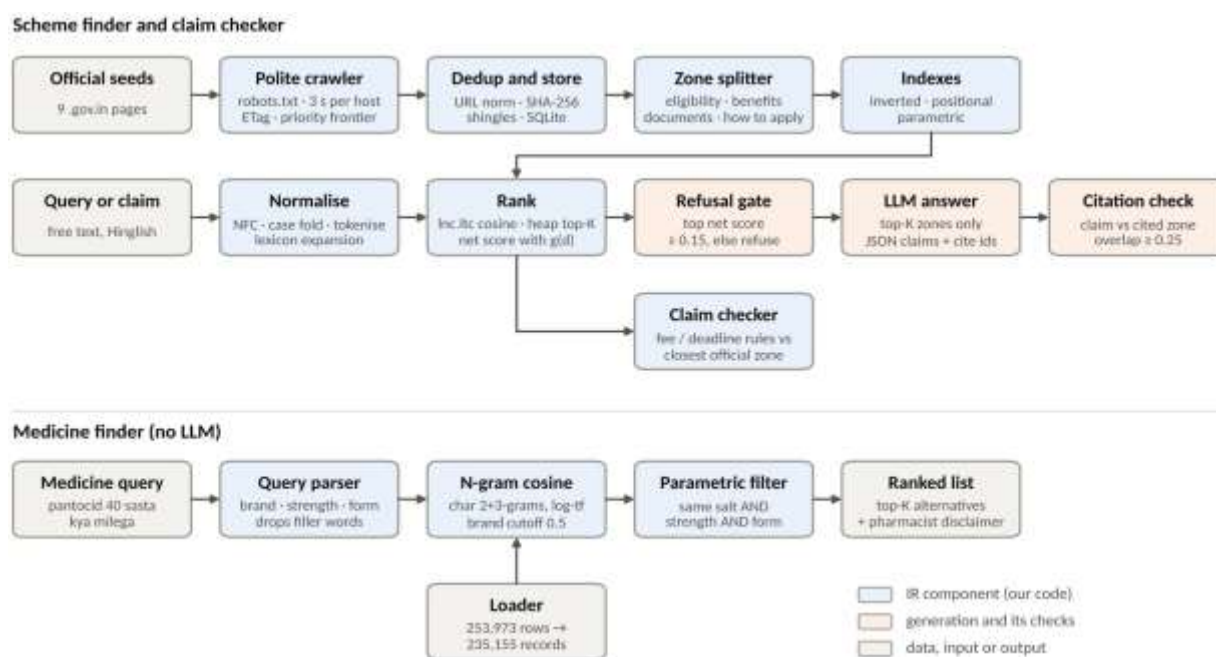


Figure 1. Bharosa pipeline. The two branches share text normalisation but use different retrieval models.

2.2 Corpus and the unit of retrieval

Scheme corpus. We crawled nine official Government of India pages ourselves: five from `socialjustice.gov.in` (AVYAY, geriatric caregivers, NAPDDR, NAMASTE, SMILE), three from `wcd.gov.in` (POSHAN 2.0, Mission Vatsalya, Mission Shakti) and one from `dghs.mohfw.gov.in`. The corpus is small on purpose. Every page is a verified official source, and each page is split by its headings into zones. The zone, not the page, is the document: users ask about one zone at a time, a zone is short enough for a claim to be checked against it, and zone type becomes a field we can filter and weight.

Medicine corpus. A static CSV of 253,973 Indian pharmaceutical products with 15 columns, sourced from the **Kaggle Indian Pharmaceutical & Medicine Catalog Dataset** and licensed under **CC0 1.0 Universal (Public Domain Dedication)**. Each row is a document with fields for brand, salt, strength, dosage form, pack and price. The loader normalises strength (650mg, 650 MG) and dosage form (tab, tablet), and accepts **235,155** records. It drops 18,818: 10,756 with an unparseable strength, 7,905 discontinued, 153 duplicates and 4 with an invalid price. A missing strength is never guessed. Combination medicines are flagged and keep no single aggregate strength.

2.3 Text processing and indexes

One tokeniser is used at index time and query time: Unicode NFC normalisation, case folding, punctuation to token boundaries and optional stop-word removal. Devanagari vowel signs and virama stay inside the token, so mixed-script text is not broken apart. The **inverted index** stores a dictionary and postings of (document, tf) and exposes df, collection frequency and $\text{idf}(t) = \log_{10}(N / \text{df})$. A **positional index** stores token positions, so a quoted query such as "Mission Vatsalya" matches only where the words are adjacent and in order. A **parametric index** holds state, condition and zone type; its constraints are intersected as a Boolean AND before ranking. **Query expansion** is a one-hop lookup in a reviewed Hinglish lexicon. It appends canonical terms and never guesses a translation.

2.4 Ranking and result assembly

Scheme zones are ranked with SMART **Inc.ltc**. The document vector uses $1 + \log_{10}(\text{tf})$ with cosine normalisation and no idf. The query vector uses $(1 + \log_{10}(\text{tf})) \times \text{idf}$ with cosine normalisation. We chose it because idf on the query side is enough to down-weight common words, and document vectors then do not change when the collection grows. The **top-K** results are selected with a min-heap of size K, not by sorting every scored zone. A **net score** then combines relevance with query-independent evidence:

$$\text{net}(d, q) = w_c \cdot \text{cosine}(d, q) + w_g \cdot g(d) + w_f \cdot \text{freshness}(d), \quad \text{freshness}(d) = 1 / (1 + \text{age_days} / \text{scale})$$

Here $g(d)$ is the static quality of the source, set per seed, and freshness is computed from the page's last detected change. An optional zone weight multiplies the sum. Each term is a command-line flag, so each can be removed for an ablation. Weights are supplied by the caller and are not fitted. **BM25** [3], through the `rank_bm25` library with $k1 = 1.5$ and $b = 0.75$, is kept as a separate baseline and never receives a net score.

2.5 Medicine retrieval

Medicine search has two stages and no LLM. The parser reads brand, strength, unit and form from the query and drops Hinglish filler words such as *sasta*, *kya* and *milega*. Stage one compares the query brand with every corpus brand using **character 2-grams and 3-grams**, log-tf weights and cosine similarity, with a cutoff of 0.5. N-grams tolerate the spelling variation that defeats exact match: `pantosid` still shares most of its grams with `pantocid`. Stage two is a **parametric filter**: a candidate is kept only if its salt, strength value, strength unit and dosage form all agree. Textual similarity alone is not safe, because similarly spelled brands can contain different salts. **Soundex**, a syllabus technique, and normalised exact match are implemented as baselines.

2.6 Crawling, duplicates and freshness

The crawler is a standard crawl loop with a **priority frontier** and one queue per host. Before each fetch it checks `robots.txt` (cached per host) and waits at least 3 seconds between requests to the same host. If `robots.txt` cannot be read, the URL is skipped. It sends a named User-Agent and conditional requests (ETag and If-Modified-Since), so unchanged pages return HTTP 304. **URL normalisation** lower-cases the host, removes fragments and tracking parameters and sorts query parameters. The **"content seen?"** test is a SHA-256 hash of the page, and **near-duplicates** are found with 5-word shingles and Jaccard similarity at a threshold of 0.8 [6]. Every version is stored in SQLite. For **freshness**, an adaptive scheduler halves a page's recrawl interval when it changes and multiplies it by 1.5 when it does not, bounded between 5 minutes and 24 hours.

IR principle (lecture topic)	Where in the code	Why we chose it
Inverted index: postings, tf, df, idf	<code>index/inverted.py</code>	From scratch, so postings can be printed
Positional index, phrase queries	<code>index/positional.py</code>	Exact names such as "Mission Vatsalya"
Tokenisation, normalisation, case folding	<code>text/normalize.py</code>	One tokeniser for index and query
Zone and parametric indexes	<code>text/zones.py</code> , <code>index/zones.py</code> , <code>index/params.py</code>	Zones are the retrieval unit and the RAG chunk; filters by state, condition and zone
tf-idf, vector space, Inc.ltc, cosine	<code>rank/tfidf.py</code>	Transparent ranker, per-term weights shown
Heap-based top-K	<code>rank/topk.py</code>	Result assembly without a full sort
Static quality $g(d)$, net score	<code>rank/netscore.py</code>	Authority and freshness added to cosine
Dictionary query expansion	<code>text/hinglish.py</code>	Controlled Hinglish expansion, reviewed lexicon
Character n-grams; Soundex	<code>medicine/ngram.py</code> , <code>medicine/phonetic.py</code>	Misspelled brands; Soundex is the syllabus baseline
Parametric (fielded) filtering	<code>medicine/parametric.py</code>	Same salt AND strength AND form
Crawl loop, frontier, politeness, robots.txt	<code>crawl/run.py</code> , <code>frontier.py</code> , <code>fetcher.py</code>	A real corpus, collected ethically
URL normalisation, content-seen, shingles	<code>crawl/dedup.py</code> , <code>crawl/shingles.py</code>	No duplicate or near-duplicate documents
Freshness and recrawl scheduling	<code>crawl/scheduler.py</code>	Spend requests where pages change
Beyond syllabus: BM25	<code>rank/bm25.py</code>	Stronger lexical baseline for scheme search

Table 2. IR principles used, where they live under `bharosa/`, and why. A function-level map is in Appendix A.

3. Beyond IR

Retrieval-gated generation. The LLM is the last step and depends entirely on the IR output. Before any call, a refusal gate checks the ranked zones: if there are no hits, if the evidence is empty, or if the top net score is below 0.15, the system answers *"not found in official sources"* and the LLM is never called. Otherwise the prompt contains only the top-K zones, labelled Z1, Z2 and so on. The model must return JSON with one claim per sentence and a list of cited zone identifiers. Output that does not parse is rejected, and a citation to a zone that was not retrieved is flagged. OpenAI, Anthropic and OpenRouter are supported through environment variables; without a key, everything except the generated answer still runs.

Citation checking. Each claim is scored against the zone it cites by lexical overlap: the fraction of the claim's tokens that occur in the zone. Below 0.25 the claim is flagged as unsupported, and a missing or unknown citation is flagged separately. This is a deliberately simple, inspectable heuristic. It is not entailment and we do not present it as a factuality model.

Claim checking. For a pasted message, pattern rules extract fee amounts, "free" statements and deadlines. The message is used as a query to retrieve the closest official zone, and deterministic rules compare the two to give *supported*, *contradicted* or *insufficient evidence*. An LLM may only reword the explanation after the verdict is fixed, so it cannot decide what is true.

Interfaces. A command-line interface with `--verbose` traces is the main interface, and a small web UI over a standard-library HTTP server is included. The frontend is not part of our claims.

4. Novelty and Creativity

The components are established techniques, and we do not claim a new ranking algorithm. What is new against the obvious baseline, a chatbot over a vector store, and against existing tools is how the components are arranged around trust:

- **Generation is assigned by risk, not applied everywhere.** Three modules receive three different amounts of LLM: none for medicine facts, constrained and checked for scheme answers, and wording-only for claim verdicts.
- **Safe substitution is a retrieval problem with a hard constraint.** Approximate matching finds the brand, and a fielded Boolean filter decides equivalence. Pharmacy apps do this behind a catalogue search; we do it for misspelled Hinglish queries, with every score visible.
- **Zones serve as index unit, parametric field, ranking feature and citation target** at the same time, which is what makes sentence-level tracing cheap.
- **A refusal path that costs nothing.** Low-confidence retrieval stops before the LLM, so the system can say "I don't know" without a model call.
- **Claims in forwarded messages are checked against a corpus we crawl ourselves** from official sources, with source quality in the score.
- **Evaluation that refuses to invent numbers.** Where judged data is missing, our evaluation script writes N/A and the reason, and AI-drafted labels are excluded from every metric.

5. Evaluation

Every number in this section is written by `python eval/run_eval.py` or `python eval/freshness_eval.py` into `eval/results/`. Nothing is entered by hand and no threshold was tuned on the evaluation queries.

5.1 Medicine finder against two baselines

We wrote 29 medicine queries. Fifteen are verified against the dataset and scored. The other 14, which include the misspelled and Hinglish variants, are still marked as needing human review and are excluded, so these results are **not** yet evidence of typo or Hinglish robustness. We use two definitions of relevance. *Brand* relevance asks whether the system found the brand the user named. *Identity* relevance asks what the user needs for substitution: a row with the same salt, strength, unit and dosage form.

Method	Brand top-1	Identity top-1	Identity P@5	Identity P@10	Identity R@10	Rows returned
Exact match (normalised brand)	0.533	0.533	0.107	0.053	0.14%	0 or 1
Soundex	0.867	0.867	0.240	0.127	3.70%	24 to 499
Ours: n-gram cosine + parametric	0.267	0.400	0.400	0.400	2.58%	at most 10

Table 3. Medicine retrieval on 15 verified queries over 235,155 records (macro average). Best value per column in bold.

Query	Soundex rows	Our rows	Our P@5	Query	Soundex rows	Our rows	Our P@5
Calpol 500mg Tablet	108	10	1.00	Dolo 650 Tablet	91	0	0.00
Azithral 500 Tablet	499	10	1.00	Dolo 650	91	0	0.00
Aciloc 150 Tablet	256	10	1.00	Avil 25	45	0	0.00
Azithral 500	499	10	1.00	Zifi 200	162	0	0.00
Allegra 120	101	10	1.00	PAN 40	99	0	0.00
Aciloc 300	256	10	1.00	Augmentin 625 Duo Tablet	150	0	0.00
Allegra-M Tablet	101	2	0.00	Combiflam Tablet	24	0	0.00
				Ascoril LS Syrup	80	0	0.00

Table 4. Per-query outcome on the 15 scored queries: rows returned by Soundex and by our method, and our identity P@5.

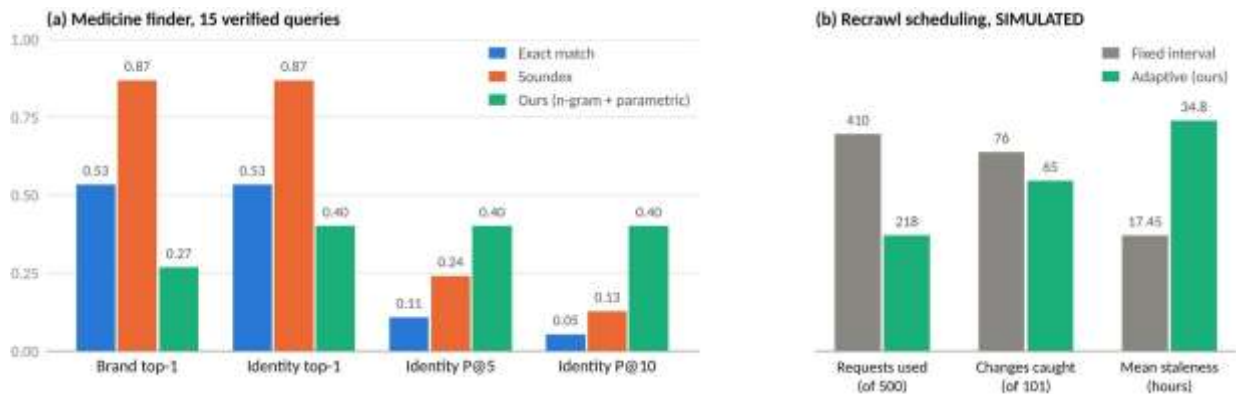


Figure 2. (a) Medicine finder against baselines. (b) Fixed and adaptive recrawl scheduling in a labelled simulation; each pair of bars has its own scale.

Reading the result. Soundex is the best at finding the named brand, but it cannot rank: it returns between 24 and 499 rows that share a code, in corpus order, and its set precision for the brand is 0.011. Exact match returns at most one row, so its P@5 cannot exceed 0.20. Our method is the only one that returns a short ranked list of true substitutes. On 6 of the 15 queries every one of its top 5 results has the same salt, strength and form (P@5 = 1.0). On 8 queries it returns nothing, and on 1 it returns two wrong rows (Table 4). Its mean P@5 of 0.40 is therefore the product of perfect precision and 40% coverage. Recall@10 is low for every method because each identity has hundreds of rows in the corpus (541 for paracetamol 650 mg tablets), so a list of ten cannot recall many of them.

Why it abstains. The failures have two causes, both visible in the verbose trace. First, when the strength is part of the brand name, as in *Dolo 650*, the parser removes 650 as the strength, and the remaining *dolo* scores below the 0.5 cutoff against *dolo 650 tablet*. Second, combination medicines such as *Augmentin 625 Duo* have no single strength and are skipped by the filter. The cutoff was a fixed design value and was not tuned.

5.2 Recrawl scheduling (simulated)

No crawled page changed during the 36 hours (9 pages, 9 versions, 0 changes), so a real freshness measurement was impossible. We therefore ran a simulation and label it as one: 10 URLs, 168 hours, 101 synthetic changes, a budget of 500 requests per scheduler and a fixed random seed. The fixed-interval scheduler used 410 requests and caught 76 of 101 changes (75.2%), with a mean staleness of 17.5 hours. The adaptive scheduler used 218 requests and caught 65 (64.4%), with a mean staleness of 34.8 hours. Adaptive is 61% more efficient per request (0.298 against 0.185 changes per request) and makes 47% fewer requests, but it is staler. This is a politeness-against-freshness trade-off, not a win, and it says nothing yet about real government pages.

5.3 Sanity checks for the parts without judged data

- **Unit tests.** 350 tests pass (1 skipped) in about two seconds with the listed requirements installed. They use small hand-made examples with known answers for every module: postings, phrase matches, Inc.Itc weights, heap order, net-score arithmetic, URL normalisation, shingles, scheduler bounds, refusal and citation flags.
- **RAG mechanism on fixtures.** With hand-written zones and a fake LLM, the checker flags 1 of 3 generated sentences as unsupported and all 3 out-of-scope queries are refused. The output file labels itself `MOCK_FIXTURE`. It shows the gate and the checker work; it is not a quality number.
- **Live traces.** On the submitted code, `pantocid 40 sasta kya milega` returns five pantoprazole 40 mg tablets (best brand cosine 0.72), and the misspelled `pantosid 40` also returns only pantoprazole 40 mg tablets, where exact match returns nothing. These are illustrations, not scored results.

5.4 Not yet measured

Scheme retrieval P@k and recall, the net-score ablations (zone weights, g(d), freshness, Hinglish expansion), RAG answer correctness and claim-checker precision need human-judged queries and labels that we did not finish. The five claim messages in the repository are AI-drafted and unreviewed, so they are not scored. For all of these the evaluation script reports N/A with the reason. We make no claim that net score, zone weighting or BM25 improves scheme retrieval.

6. Limitations and Next Steps

- **Evaluation coverage is the main weakness.** 15 scored medicine queries, none of them misspelled or Hinglish, and no judged scheme or RAG data.
- **The medicine finder abstains too often.** Short brands, brands with the strength in the name, and combination medicines return nothing. An unknown filler word also breaks the brand: `calpol 500 sasta option kya milega` keeps `option` and falls to a cosine of 0.48.
- **Medicine search is slow**, about 30 to 50 seconds on the command line, because the query is scored against all 235,155 brands with no candidate pruning.
- **Results are ranked by brand similarity, not price.** MRP is a full-pack price, and the dataset has no generic or Jan Aushadhi price. The dataset is a static snapshot.
- **Hinglish expansion for scheme search needs a reviewed lexicon**, which is not committed yet, and there is no transliteration between Devanagari and Latin spellings.
- **The scheme corpus is nine pages**, net-score weights are set by hand, and the freshness result is simulated.
- **The citation checker is lexical.** A claim that reuses the zone's words in a wrong combination can pass.

Roadmap. We plan to continue Bharosa as our course project, in this order:

1. A k-gram inverted index over brand names to prune candidates, and a combined n-gram and Soundex score with a tuned cutoff. Figure 2(a) suggests the two are complementary: Soundex finds the brand, the parametric filter makes the list safe.
2. A benchmark of 40 or more medicine queries with reviewed misspelled and Hinglish variants, and 30 or more scheme queries with relevance judgements from two annotators.
3. Scheme ablations: Inc.Itc, BM25, and net score with and without zone weights, g(d), freshness and Hinglish expansion.
4. A reviewed Hinglish lexicon and Devanagari transliteration.
5. Hybrid sparse and dense retrieval with an analysis of when each wins, and a cosine or entailment-based citation checker judged by humans.
6. Jan Aushadhi prices with per-unit price ranking, and a larger crawl with stored snapshots to replace the simulated freshness result.

7. Work Division

Member	Worked on
Parth Agarwal	Crawler: fetcher, frontier, robots.txt and politeness, URL and content deduplication, shingles, adaptive scheduler, verified seeds, freshness evaluation; RAG answer, citation checker and claim checker
Paridhi	Text normalisation, inverted, positional and parametric indexes, Inc.Itc ranking, heap top-K, net score, Hinglish expansion, scheme and medicine search, CLI, API, evaluation pipeline
Ishanvi Singh	Medicine dataset, loader and data audit, Soundex and exact-match baselines, medicine evaluation queries, web frontend
Kushagra Bansal	RAG, claim checking, and evaluation: Implemented the LLM-based cited-answer pipeline, confidence-based refusal, citation/claim verification, and the evaluation framework.

8. AI-Use Declaration

AI tools were used throughout, as the assignment allows. **Cursor** (AI coding agent) was used to scaffold and draft code and unit tests in every module; the team reviewed and tested the final implementation. **An LLM API (OpenRouter, using meta-llama/llama-3.3-70b-instruct)** is used inside the system only to write scheme answers from retrieved zones and, optionally, to reword claim explanations. It is never used for medicine facts. **Claude** (Anthropic) was used to review the repository and to draft the README, the demo script and this report from the code and the stored evaluation files. Some evaluation rows were AI-drafted: 14 medicine queries and 5 claim messages are marked as unreviewed in the repository and are excluded from all metrics. No result in this report was generated by an AI model; every number comes from running the evaluation scripts on the submitted code.

References

- [1] G. Salton, A. Wong and C. S. Yang. A vector space model for automatic indexing. *Communications of the ACM*, 18(11), 1975.
- [2] C. D. Manning, P. Raghavan and H. Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [3] S. Robertson and H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 2009.
- [4] P. Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS*, 2020.
- [5] T. Gao, H. Yen, J. Yu and D. Chen. Enabling large language models to generate text with citations. *EMNLP*, 2023.
- [6] A. Z. Broder. On the resemblance and containment of documents. *Compression and Complexity of Sequences*, 1997.
- [7] A. Heydon and M. Najork. Mercator: a scalable, extensible web crawler. *World Wide Web*, 2(4), 1999.
- [8] J. Cho and H. Garcia-Molina. Effective page refresh policies for web crawlers. *ACM Transactions on Database Systems*, 28(4), 2003.

Data and libraries. Scheme pages: Government of India portals listed in Section 2.2, crawled by us in compliance with robots.txt with a minimum 3-second delay; per-seed checks are documented in docs/seeds.md. Medicine records: indian_pharmaceutical_products_clean.csv, a static dataset of 253,973 Indian pharmaceutical products across 15 columns, sourced from the Kaggle Indian Pharmaceutical & Medicine Catalog Dataset and licensed under CC0 1.0 Universal (Public Domain Dedication). Libraries: `requests` (HTTP fetching), `pyyaml` (seed file), `rank_bm25` (BM25 baseline only), `pytest` (tests). HTML parsing and the web server use the Python standard library.

Appendix A: Implementation Map

IR principle	File and functions (under <code>bharosa/</code>)	Role
Tokenisation, normalisation	<code>text/normalize.py: normalize(), tokenize()</code>	NFC, case folding, token boundaries, optional stop words
Inverted index	<code>index/inverted.py: add_document(), postings(), document_frequency(), idf()</code>	Dictionary and postings (doc, tf)
Positional index	<code>index/positional.py: add_document(), find_phrase()</code>	Exact phrase matching
Parametric index	<code>index/params.py: matching_ids()</code>	Boolean AND over state, conditions, zone
tf-idf and cosine	<code>rank/tfidf.py: document_vector(), query_vector(), cosine(), rank()</code>	SMART Inc.ltc with L2 normalisation
Heap top-K	<code>rank/topk.py: top_k()</code>	Min-heap of at most K candidates
Zone extraction and search	<code>text/zones.py: load_zone_documents(); index/zones.py: search_schemes()</code>	Pages to zones; text, phrase and field filters
Net score	<code>rank/netscore.py: NetScorer, NetScoreConfig</code>	Cosine + g(d) + freshness, zone weight
BM25 baseline	<code>rank/bm25.py: BM25Baseline</code>	Wraps <code>rank_bm25.BM25Okapi</code>
Query expansion	<code>text/hinglish.py: HinglishLexicon.expand()</code>	One-hop reviewed-lexicon expansion
Character n-grams	<code>medicine/ngram.py: character_ngrams(), ngram_similarity()</code>	2-gram + 3-gram log-tf cosine
Medicine constraints	<code>medicine/parametric.py: passes_query_constraints()</code>	Salt, strength, unit, form
Medicine search	<code>medicine/search.py: parse_medicine_query(), search_medicine()</code>	Two-stage retrieval, cutoff 0.5
Baselines	<code>medicine/phonetic.py: soundex(); medicine/baselines.py</code>	Soundex and exact brand match
Frontier, politeness	<code>crawl/frontier.py: CrawlFrontier; crawl/fetcher.py: RobotsCache, Fetcher</code>	Priority queue, robots.txt, 3 s delay, ETag
URL and content dedup	<code>crawl/dedup.py: normalize_url(), compute_content_hash()</code>	Canonical URLs, SHA-256
Near-duplicates	<code>crawl/shingles.py: get_word_shingles(), jaccard_similarity()</code>	5-word shingles, threshold 0.8
Recrawl scheduling	<code>crawl/scheduler.py: FixedIntervalScheduler, AdaptiveScheduler</code>	Interval x0.5 on change, x1.5 otherwise
RAG gate and answer	<code>rag/answer.py: should_refuse(), build_prompt(), parse_llm_answer()</code>	Refuse below 0.15; zones-only prompt
Citation check	<code>rag/citations.py: check_citations()</code>	Lexical overlap, threshold 0.25
Claim check	<code>rag/claimcheck.py: extract_claim_spans(), compare_claim(), check_claim()</code>	Deterministic verdicts

Table A1. Function-level implementation map.

Appendix B: Reproducing the Results

```

pip install -r requirements.txt
python -m pytest -q
python -m bharosa.crawl.run --seeds config/seeds.yaml --max-pages 50 --verbose
python -m bharosa.app.cli medicine "pantocid 40 sasta kya milega" --medicine-file
data/medicines/indian_pharmaceutical_products_clean.csv --verbose
python -m bharosa.app.cli scheme "health insurance for sanitation workers" -k 5 --g-score --verbose
python -m bharosa.app.cli claim "pay Rs 500 to activate free health card" --verbose
python eval/run_eval.py
python eval/freshness_eval.py

```